

Guía Práctica: Simulación de Smart Meters

1. Fundamento Matemático: El Modelo de Mezcla de Gaussianas (GMM)

El consumo de agua en un hogar no es constante ni puramente aleatorio. Responde a hábitos humanos altamente predecibles distribuidos en tres franjas horarias principales (picos de consumo):

1. **Mañana (Ducha y desayuno):** ~07:00 a 09:00.
2. **Mediodía (Comida):** ~13:30 a 15:00.
3. **Noche (Cena y aseo):** ~20:00 a 22:30.

Para modelar este comportamiento de forma continua, utilizamos una **Mezcla de Gaussianas** (GMM, *Gaussian Mixture Model*). Una distribución de mezcla combina varias funciones de densidad de probabilidad normales (Gaussianas), cada una con su propio peso, media (hora punta) y desviación estándar (duración del pico).

La Ecuación de la Demanda Teórica

La función de densidad de probabilidad (PDF) del consumo a lo largo del día ($t \in [0, 24]$ horas) se define como:

$$P(t) = \sum_{k=1}^K w_k \cdot \mathcal{N}(t; \mu_k, \sigma_k^2)$$

Donde:

- $K = 3$ es el número de picos o componentes de la mezcla.
- w_k es el **peso** de la componente k , cumpliendo que $\sum_{k=1}^K w_k = 1$ para que sea una distribución de probabilidad válida.
- $\mathcal{N}(t; \mu_k, \sigma_k^2)$ es la distribución normal, definida como:

$$\mathcal{N}(t; \mu_k, \sigma_k^2) = \frac{1}{\sigma_k \sqrt{2\pi}} e^{-\frac{(t-\mu_k)^2}{2\sigma_k^2}}$$

•

μ_k es la **hora central** del pico de consumo k .

- σ_k es la **desviación estándar** (la dispersión o "ancho" temporal del pico de consumo).

Inclusión de Pérdidas por Fuga (Ruido Base)

Para modelar pérdidas continuas (fugas) o el consumo mínimo residual nocturno, añadimos una probabilidad basal constante P_{base} . Así, la probabilidad final de consumo en un instante t se comporta como:

$$P_{\text{total}}(t) = P(t) \cdot I_{\text{familiar}} + P_{\text{base}}$$

Donde I_{familiar} es un multiplicador aleatorio asignado a cada hogar para simular familias numerosas (alto consumo) o personas solas (bajo consumo).

2. De la Probabilidad Continua al "Tick" de Simulación (Proceso de Bernoulli)

Los contadores de agua digitales reales (emisores de pulsos) funcionan discretizando el volumen: cada vez que pasa un litro de agua por el sensor, este emite un pulso o señal digital (evento).

Para simular esto en un ordenador segundo a segundo (o minuto a minuto), realizamos un **Ensayo de Bernoulli** en cada "tick" de tiempo.

1. Definimos un paso de tiempo Δt (ej. $\Delta t = 1$ minuto o 1 segundo).
2. Evaluamos la probabilidad continua $P_{\text{total}}(t)$ en el instante actual.
3. Escalamos esta probabilidad al intervalo temporal Δt :

$$P_{\text{tick}} = P_{\text{total}}(t) \cdot \Delta t$$

4.

Generamos un número pseudoaleatorio uniforme $r \in [0, 1)$.

5. Si $r < P_{\text{tick}}$, el contador emite un **pulso (1)**. En caso contrario, permanece en **silencio (0)**.

Desincronización de Contadores (El Factor Humano)

Si todos los contadores usaran exactamente la misma fórmula matemática, todos los hogares de la ciudad emitirían pulsos exactamente en el mismo microsegundo, lo cual es irreal.

Para solucionarlo, cada contador se instancia con un sesgo individual:

- **Desfase horario (δ_{offset}):** Un valor aleatorio entre -45 y $+45$ minutos que desplaza los hábitos del hogar (unos madrugan más, otros cenan más tarde).
- **Intensidad de consumo (I_{familiar}):** Un factor de escala que incrementa o reduce la probabilidad global de generar pulsos.

3. Implementación en Python y Visualización Gráfica

El siguiente código implementa el generador probabilístico y utiliza matplotlib y seaborn para pintar un panel de diagnóstico de alta calidad.

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
import random
```

```
class ContadorAguaVisual:
```

```
    def __init__(self, contador_id):
```

```
        self.contador_id = contador_id
```

```
        # Parámetros de los 3 picos: [Peso (w), Media (mu), Desviación (sigma)]
```

```
        self.picos = [
```

```
            [0.4, 8.0, 1.0], # Mañana (8:00 AM)
```

```
            [0.2, 14.5, 1.2], # Mediodía (2:30 PM)
```

```
            [0.4, 21.0, 1.5] # Noche (9:00 PM)
```

```
        ]
```

```
        # Desfase individual del hogar en horas (entre -0.75 y +0.75 horas -> +/- 45 min)
```

```
        self.offset_individual = random.uniform(-0.75, 0.75)
```

```
        # Consumo basal (fugas continuas o goteos)
```

```
        self.probabilidad_base = 0.002
```

```
        # Factor de escala de consumo del hogar
```

```
        self.intensidad_base = random.uniform(0.6, 1.4)
```

```
    def calcular_probabilidad(self, hora):
```

```
        """Calcula la densidad de probabilidad para una hora específica del día (0-24)"""
```

```
        # Aplicamos el desfase individual de forma cíclica en formato 24h
```

```
        hora_ajustada = (hora + self.offset_individual) % 24
```

```
        prob_picos = 0
```

```
        for w, mu, sigma in self.picos:
```

```
            # Ecuación de la distribución normal (Gaussiana)
```

```

    exponente = -0.5 * ((hora_ajustada - mu) / sigma) ** 2
    gaussiana = (1 / (sigma * np.sqrt(2 * np.pi))) * np.exp(exponente)
    prob_picos += w * gaussiana

    return (prob_picos * self.intensidad_base) + self.probabilidad_base

def simular_dia(self, intervalo_minutos=1):
    """Simula un día completo minuto a minuto y retorna las horas de los pulsos generados"""
    pulsos_detectados = []
    pasos_totales = int(24 * (60 / intervalo_minutos))
    horas_dia = np.linspace(0, 24, pasos_totales)

    ticks_por_hora = 60 / intervalo_minutos

    for hora in horas_dia:
        prob_hora = self.calcular_probabilidad(hora)
        # Escalamos la probabilidad horaria al tamaño del tick temporal
        prob_tick = prob_hora / ticks_por_hora

        # Proceso de Bernoulli (Lanzamiento de moneda sesgada)
        if random.random() < prob_tick:
            pulsos_detectados.append(hora)

    return pulsos_detectados

# =====
# CONFIGURACIÓN Y EJECUCIÓN DE LA SIMULACIÓN
# =====

sns.set_theme(style="whitegrid")
plt.rcParams.update({'font.size': 11, 'figure.titlesize': 16})

num_contadores = 30
contadores = [ContadorAguaVisual(f"Contador-{i+1:02d}") for i in range(num_contadores)]

todos_los_pulsos = {}
lista_global_de_pulsos = []

for c in contadores:
    pulsos = c.simular_dia(intervalo_minutos=1) # Simulación minuto a minuto
    todos_los_pulsos[c.contador_id] = pulsos
    lista_global_de_pulsos.extend(pulsos)

```

```

# =====
# GENERACIÓN DE GRÁFICOS (3 PANELES)
# =====

fig, axes = plt.subplots(3, 1, figsize=(12, 12), sharex=True,
                        gridspec_kw={'height_ratios': [1.5, 3, 2]})

# --- PANEL 1: Curva Teórica de Probabilidad ---
eje_x_horas = np.linspace(0, 24, 500)
contador_teorico = ContadorAguaVisual("Teórico")
contador_teorico.offset_individual = 0 # Sin desfase para ver el patrón limpio
contador_teorico.intensidad_base = 1.0
y_teorico = [contador_teorico.calcular_probabilidad(h) for h in eje_x_horas]

axes[0].plot(eje_x_horas, y_teorico, color='#1f77b4', lw=2.5, label='Mezcla de 3 Gaussianas (GMM)')
axes[0].fill_between(eje_x_horas, y_teorico, alpha=0.15, color='#1f77b4')
axes[0].set_title("1. Perfil de Probabilidad Teórica de Consumo Diario (GMM)", loc='left',
fontweight='bold')
axes[0].set_ylabel("Densidad Prob.")
axes[0].legend()

# --- PANEL 2: Raster Plot (Pulsos individuales desincronizados) ---
for idx, (cid, pulsos) in enumerate(todos_los_pulsos.items()):
    axes[1].scatter(pulsos, [idx] * len(pulsos),
                    color='#e74c3c', alpha=0.6, s=15, marker='|')

axes[1].set_title("2. Registro de Pulsos Individuales (Simulación de 30 Contadores)", loc='left',
fontweight='bold')
axes[1].set_ylabel("ID del Contador")
axes[1].set_yticks(range(num_contadores))
axes[1].set_yticklabels([f"ID {i+1:02d}" for i in range(num_contadores)], fontsize=8)
axes[1].set_ylim(-1, num_contadores)

# --- PANEL 3: Histograma Acumulado (La Realidad Reconstruida) ---
sns.histplot(lista_global_de_pulsos, bins=48, kde=True, ax=axes[2],
            color='#2ecc71', edgecolor='white', alpha=0.7)
axes[2].set_title("3. Volumen de Consumo Agregado Registrado (Suma de todos los pulsos)",
loc='left', fontweight='bold')
axes[2].set_xlabel("Hora del Día (00:00 - 24:00)")
axes[2].set_ylabel("Pulsos Emitidos (Litros)")
axes[2].set_xlim(0, 24)
axes[2].set_xticks(range(25))

```

```
plt.tight_layout()
plt.show()
```

Explicación pedagógica de los gráficos:

- **El primer panel** enseña la teoría estadística limpia e ideal.
- **El segundo panel (Raster Plot)** demuestra el caos del mundo real: los eventos son discretos y desordenados. Cada contador emite a horas ligeramente distintas.
- **El tercer panel** es la demostración empírica de la **Ley de los Grandes Números**: al sumar (agregar) el comportamiento caótico y ruidoso de miles de hogares, la forma de la curva que emerge coincide exactamente con la probabilidad teórica del primer panel.

4. Paralelismo y Pruebas de Carga (Generación de Miles de Pulsos por Segundo)

En entornos de producción reales (como las plataformas IoT urbanas), el sistema de ingesta debe ser capaz de procesar miles de pulsos por segundo provenientes de toda una ciudad. Para que los alumnos aprendan a estresar un sistema de ingesta o simular miles de sensores en tiempo real de forma eficiente, introducimos el uso de **programación concurrente/paralela en Python**.

Estrategia de Paralelización

Utilizaremos el módulo `concurrent.futures.ProcessPoolExecutor` para distribuir la simulación de múltiples contadores a través de los diferentes núcleos físicos de la CPU (evitando el bloqueo del GIL de Python).

El siguiente script simula la generación masiva de pulsos para miles de contadores en paralelo, agrupando los datos en formato JSON simulando una trama de red IoT real que podría enviarse a un servicio de mensajería (como Kafka, MQTT o una API HTTP).

```
import time
from concurrent.futures import ProcessPoolExecutor
import numpy as np
import random
```

```
# Definición de la clase optimizada para cálculo rápido
```

```
class ContadorIoT:
```

```
    def __init__(self, id_contador):
        self.id_contador = id_contador
        self.offset = random.uniform(-0.75, 0.75)
        self.intensidad = random.uniform(0.5, 1.5)
        self.fuga = random.choice([0.0, 0.0, 0.0, 0.05]) # 25% probabilidad de tener fuga real
```

```

def simular_bloque_horas(self, hora_inicio, hora_fin, ticks_por_segundo=1):
    """
    Simula un tramo de tiempo y retorna los pulsos generados en formato IoT
    """
    eventos = []
    picos = [[0.4, 8.0, 1.0], [0.2, 14.5, 1.2], [0.4, 21.0, 1.5]]

    # Simulación de un bloque temporal discreto
    for t in np.arange(hora_inicio, hora_fin, 1.0 / (3600 * ticks_por_segundo)):
        hora_ajustada = (t + self.offset) % 24

        # Evaluación del GMM
        prob_gmm = 0
        for w, mu, sigma in picos:
            exponente = -0.5 * ((hora_ajustada - mu) / sigma) ** 2
            prob_gmm += w * (1.0 / (sigma * np.sqrt(2 * np.pi))) * np.exp(exponente)

        prob_final = (prob_gmm * self.intensidad) / (3600 * ticks_por_segundo)
        # Sumamos la probabilidad de fuga (si la hay) por cada segundo
        prob_final += self.fuga / (3600 * ticks_por_segundo)

        if random.random() < prob_final:
            # Almacenamos el evento simulado
            eventos.append({
                "timestamp": round(t, 4),
                "sensor_id": self.id_contador,
                "evento": "PULSO_1L"
            })
    return eventos

# Función de ejecución global que será mapeada a los procesos
def ejecutar_simulacion_contador(id_num):
    # Simula el consumo de un contador entre las 07:00 y las 09:00 AM (Franja crítica de la mañana)
    contador = ContadorIoT(id_contador=f"CNT_{id_num:06d}")
    return contador.simular_bloque_horas(7.0, 9.0, ticks_por_segundo=1)

if __name__ == '__main__':
    num_sensores = 5000 # Cantidad masiva de contadores a simular en paralelo
    print(f"Iniciando simulación paralela para {num_sensores} contadores inteligentes...")

    inicio_tiempo = time.time()

```

```

# Usamos ProcessPoolExecutor para aprovechar todos los núcleos de la CPU
with ProcessPoolExecutor() as executor:
    # Mapeamos los contadores a la función simuladora distribuyéndolos en paralelo
    resultados = executor.map(ejecutar_simulacion_contador, range(num_sensores))

fin_tiempo = time.time()

# Procesar resultados agregados
total_pulsos_generados = 0
for r in resultados:
    total_pulsos_generados += len(r)

tiempo_empleado = fin_tiempo - inicio_tiempo
print("\n--- RESULTADOS DE LA SIMULACIÓN CONCURRENTE ---")
print(f"Tiempo total de cómputo: {tiempo_empleado:.2f} segundos")
print(f"Pulsos de agua totales procesados: {total_pulsos_generados} eventos")
print(f"Rendimiento de simulación: {int(total_pulsos_generados / tiempo_empleado)} eventos/segundo")

```

¿Por qué usar Probabilidad Basal y Ensayos de Bernoulli en IoT?

1. El Concepto de "Probabilidad Basal" (Fugas y Consumo Nocturno)

El Problema del Modelo Puro

Si solo utilizáramos la Mezcla de Gaussianas (GMM) para simular el consumo, tendríamos un modelo de "comportamiento humano perfecto". La ecuación del consumo humano $P(t)$ nos daría curvas muy bonitas que bajan casi a 0 a las 03:00 AM, porque a esa hora prácticamente todo el mundo duerme.

Si la probabilidad a las 03:00 AM es matemáticamente cero ($P(3) = 0$):

4. El contador **jamás** emitiría un pulso de agua a esa hora.
5. La simulación sería artificialmente perfecta.

La Solución: La Probabilidad Basal (P_{base})

En el mundo real, la red de agua de una ciudad nunca está en silencio absoluto. Para simular esta realidad de forma matemática, sumamos un término constante llamado **probabilidad basal** o ruido de fondo:

$$P_{\text{total}}(t) = P(t) + P_{\text{base}}$$

¿Qué representa físicamente este pequeño valor constante P_{base} ?

- **Fugas continuas en el hogar:** Un grifo que gotee (un goteo constante cada pocos segundos) o una cisterna del inodoro que pierda un hilo de agua imperceptible. Esto ocurre las 24 horas del día, independientemente de si la familia duerme, come o está fuera de casa.
- **Consumo residual esporádico:** Alguien que se levanta a las 04:00 AM sediento a beber un vaso de agua, o que va al baño a mitad de la noche.

Impacto en la Simulación

- **Sin P_{base} :** Si analizamos los datos a las 03:00 AM, el consumo agregado de la ciudad sería exactamente de 0 litros.
- **Con P_{base} :** Habrá un "goteo" de eventos muy esporádico pero constante durante la noche. Esto permite que los alumnos diseñen **algoritmos de detección de fugas**: si un contador emite pulsos a las 03:00 AM de forma sistemática día tras día, es altamente probable que tenga una fuga en la instalación.

2. ¿Por qué Bernoulli? De la Probabilidad Continua al "Tick" de Simulación

Para entender por qué usamos a Jacob Bernoulli en esto, primero debemos entender cómo funciona el hardware de un contador de agua real y cómo lo representamos en un ordenador.

El Reto: Probabilidad (Flujo Continuo) vs. Pulsos (Eventos Discretos)

Nuestra fórmula matemática de consumo $P(t)$ es **continua**. Nos da un flujo constante (como abrir un grifo un poco o mucho).

Sin embargo, un contador inteligente de agua real **no mide de forma continua**. Funciona mediante un sensor de pulsos magnéticos: cada vez que la turbina interior da una vuelta

completa (lo que equivale exactamente a un volumen discreto, por ejemplo, 1 litro de agua), el contador genera un pulso eléctrico. Es decir, el flujo de datos que llega a nuestro servidor IoT son eventos discretos en momentos concretos del tiempo:

Evento en $t_1 \rightarrow$ Silencio $\rightarrow$ Silencio $\rightarrow$ Evento en t_2

¿Cómo convertimos una probabilidad continua de consumo en eventos binarios de "pulso" o "silencio" segundo a segundo? Aquí es donde entra el **Ensayo de Bernoulli**.

¿Qué es un Ensayo de Bernoulli?

En teoría de la probabilidad, un **Ensayo de Bernoulli** es un experimento aleatorio donde solo hay **dos resultados posibles**:

4. **Éxito (1)**: Ocurre el evento (en nuestro caso, el contador emite un pulso porque ha pasado 1 litro de agua).
5. **Fracaso (0)**: No ocurre nada (el contador está en silencio).

La probabilidad de obtener un "Éxito" (un pulso) en un instante de tiempo es p , y la de un "Fracaso" es $1 - p$.

El Mecanismo: El Lanzamiento de la Moneda Trucada

Imagina que dividimos el día en intervalos muy pequeños (por ejemplo, segundo a segundo). Cada segundo es un "tick" de simulación.

En cada segundo t , el ordenador calcula la probabilidad de consumo de ese instante, $P_{\text{total}}(t)$. Imaginemos que a las 08:30 AM la probabilidad calculada y adaptada al segundo es del 30% ($p = 0.3$).

Para decidir si el contador emite un pulso en ese segundo exacto, el ordenador hace lo siguiente:

6. Lanza una moneda "trucada" en la que hay un 30% de probabilidad de que salga cara (Éxito) y un 70% de que salga cruz (Fracaso).
7. En código, esto se hace generando un número pseudoaleatorio r entre 0 y 1:
 - Si $r < 0.3$, la moneda cae en "cara" $\rightarrow$ **¡PULSO! (1)**.
 - Si $r \geq 0.3$, la moneda cae en "cruz" $\rightarrow$ **Silencio (0)**.

¿ $r < p$?

└─ Sí ($r = 0.12$) $\rightarrow$ ÉXITO (Se emite pulso de 1L)
└─ NO ($r = 0.74$) $\rightarrow$ FRACASO (Silencio, no hay consumo)

¿Por qué es este el método correcto?

Si repites este "lanzamiento de moneda" (Ensayo de Bernoulli) segundo tras segundo:

- A las 08:30 AM (hora punta), la probabilidad P de la moneda es alta, por lo que saldrán muchas "caras" (muchos pulsos seguidos, simulando un grifo muy abierto).
- A las 16:00 PM (hora de poco consumo), la probabilidad P de la moneda es muy baja, por lo que saldrán casi todo "cruces" (silencio, grifo cerrado).

Este comportamiento de repetir muchos ensayos de Bernoulli independientes a lo largo del tiempo se conoce en estadística como un **Proceso de Bernoulli**. Es la forma matemáticamente perfecta de transformar un modelo de comportamiento abstracto en un flujo real de eventos IoT asíncronos.